

■■ React + ■ Spring Boot + ■■ Azure SQL

Complete Azure Deployment Guide

Portal • Azure CLI • Bicep IaC • Azure DevOps Pipelines

CRUD Application • Key Vault Secrets • Application Insights •
Blue-Green Deploy

H2 for Local Development • Azure SQL for Production

■■ ReactJS 18	■ Spring Boot 3	■■ Azure SQL	■ Key Vault
■ Bicep IaC	■ Azure DevOps	■ Blue-Green	■ App Insights

Table of Contents

PART 1 — APPLICATION ARCHITECTURE & SETUP

- Ch 1: Architecture Overview & Tech Stack
- Ch 2: CRUD Application — Backend (Spring Boot)
- Ch 3: CRUD Application — Frontend (ReactJS)

PART 2 — AZURE INFRASTRUCTURE (PORTAL)

- Ch 4: Resource Group, Networking & Azure SQL
- Ch 5: Key Vault — Secrets Management
- Ch 6: App Service — Backend & Frontend
- Ch 7: Application Insights & Monitoring

PART 3 — AZURE INFRASTRUCTURE (CLI)

- Ch 8: Full CLI Deployment Script — Step by Step

PART 4 — BICEP INFRASTRUCTURE AS CODE

- Ch 9: Bicep Template — Full Explanation
- Ch 10: Deploy & Manage with Bicep

PART 5 — AZURE DEVOPS PIPELINES

- Ch 11: Pipeline Setup — Service Connections & Variable Groups
- Ch 12: React Frontend CI/CD Pipeline — Deep Dive
- Ch 13: Spring Boot Backend CI/CD Pipeline — Deep Dive
- Ch 14: Blue-Green Deployment Strategy

PART 6 — CONFIGURATION & SECRETS

- Ch 15: Key Vault Integration — Portal, CLI & Code
- Ch 16: Application Configuration — Profiles Deep Dive

PART 7 — VERIFICATION & TROUBLESHOOTING

- Ch 17: End-to-End Testing & Verification
- Ch 18: Common Issues & Fixes

PART 1

Application Architecture & Setup

Understanding and running the CRUD application locally

Ch 1: Architecture Overview

Ch 2: Spring Boot Backend

Ch 3: React Frontend

CHAPTER 1

Architecture Overview & Tech Stack

What we're building, how it fits together, and why each piece is chosen.

1.1 The Application: Employee Management System

We are building a full-stack CRUD (Create, Read, Update, Delete) web application for managing employee records. The application follows a classic 3-tier architecture: a React frontend, a Spring Boot REST API backend, and an Azure SQL relational database.

Layer	Technology	Runs On (Local)	Runs On (Azure)
Frontend	ReactJS 18 + MUI + React Router	localhost:3000	Azure App Service (Node 18)
Backend	Spring Boot 3 + Spring Data JPA	localhost:8080	Azure App Service (Java 17)
Database (local)	H2 In-Memory Database	Embedded	—
Database (cloud)	Azure SQL Database	—	Azure SQL (PaaS)
Secrets	Azure Key Vault	—	Key Vault + Managed Identity
Monitoring	Application Insights	—	App Insights + Log Analytics
IaC	Bicep + CLI	—	ARM deployments
CI/CD	Azure DevOps YAML Pipelines	—	Azure Pipelines

1.2 Architecture Diagram

The request flow works as follows:

- **User browser** → HTTPS → React frontend (Azure App Service, Node runtime)
- **React app** → HTTPS API calls → Spring Boot backend (Azure App Service, Java runtime)
- **Spring Boot** → JDBC → Azure SQL Database (via private firewall rule)
- **Spring Boot** → Managed Identity → Azure Key Vault (no passwords in config!)
- **Both App Services** → Application Insights → Log Analytics Workspace
- **Azure DevOps Pipelines** → Service Connection → Deploy to both App Services

1.3 Local Development Setup

For local development, the Spring Boot backend uses an H2 in-memory database so developers need zero external dependencies. Just clone the repo and run — the database is created automatically in RAM and seed data is loaded

from a SQL file.

Concern	Local (H2 Profile)	Azure (Azure Profile)
Database	H2 in-memory (auto-created)	Azure SQL (managed PaaS)
DB Console	http://localhost:8080/h2-console	Azure Portal Query Editor
Secrets	application-local.yml (plaintext OK)	Azure Key Vault (RBAC)
DDL	create-drop (fresh on restart)	update (preserves data)
Seed data	data-local.sql auto-loaded	Manual/Flyway migrations
Start command	mvnw spring-boot:run -Dspring-boot.run.profiles=local	Managed by App Service

1.4 Prerequisites Checklist

- **Local dev:** Java 17+ (OpenJDK), Maven 3.8+, Node.js 18+, npm 9+, Git
- **Azure:** Azure account with active subscription (free tier works for dev!)
- **DevOps:** Azure DevOps organization (free at dev.azure.com)
- **IDE:** IntelliJ IDEA or VS Code with Java Extension Pack
- **Tools:** Azure CLI 2.50+ installed and logged in (az login)

CHAPTER 2

CRUD Application — Backend (Spring Boot)

Complete Spring Boot REST API with JPA, validation, exception handling, and dual database profiles.

2.1 Project Structure

```
■ Project Structure
backend/
■ ■ ■ src/main/java/com/azure/demo/
■ ■ ■ ■ AzureDemoApplication.java # Main entry point
■ ■ ■ ■ model/
■ ■ ■ ■ ■ Employee.java # JPA Entity (@Entity, @Table)
■ ■ ■ ■ repository/
■ ■ ■ ■ ■ EmployeeRepository.java # Spring Data JPA (extends JpaRepository)
■ ■ ■ ■ service/
■ ■ ■ ■ ■ EmployeeService.java # Business logic + transactions
■ ■ ■ ■ controller/
■ ■ ■ ■ ■ EmployeeController.java # REST endpoints (@RestController)
■ ■ ■ ■ ■ HealthController.java # /api/health endpoint
■ ■ ■ ■ exception/
■ ■ ■ ■ ■ GlobalExceptionHandler.java # @RestControllerAdvice
■ ■ ■ ■ ■ ResourceNotFoundException.java
■ ■ ■ ■ ■ DuplicateEmailException.java
■ ■ ■ src/main/resources/
■ ■ ■ ■ application.yml # Base config (shared)
■ ■ ■ ■ application-local.yml # H2 profile (local dev)
■ ■ ■ ■ application-azure.yml # Azure SQL profile (production)
■ ■ ■ ■ data-local.sql # Seed data for local H2
■ ■ ■ pom.xml # Maven dependencies
```

2.2 Employee Entity (JPA Model)

The Employee JPA entity maps to the 'employees' table in the database. It uses @PrePersist and @PreUpdate lifecycle hooks to automatically set timestamps:

```
■ Employee.java (simplified)
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank @Size(max = 100)
    @Column(name = "first_name", nullable = false)
    private String firstName;
    @Email @NotBlank
    @Column(unique = true) // Email must be unique!
    private String email;
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
```

```
@PrePersist
protected void onCreate() { createdAt = updatedAt = LocalDateTime.now(); }

@PreUpdate
protected void onUpdate() { updatedAt = LocalDateTime.now(); }

// ... getters, setters
}
```

2.3 Repository — Spring Data JPA Magic

Spring Data JPA generates all SQL automatically from method names. No boilerplate SQL needed!

```
■ EmployeeRepository.java

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Spring generates: SELECT * FROM employees WHERE email = ?
    Optional<Employee> findByEmail(String email);

    // Spring generates: SELECT * FROM employees WHERE department = ?
    List<Employee> findByDepartment(String department);

    // Custom JPQL query for multi-field search
    @Query("SELECT e FROM Employee e WHERE " +
        "LOWER(e.firstName) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
        "LOWER(e.email) LIKE LOWER(CONCAT('%', :keyword, '%'))")
    List<Employee> searchByKeyword(@Param("keyword") String keyword);

    boolean existsByEmail(String email);
}
```

2.4 REST Controller — API Endpoints

HTTP Method	Endpoint	Description	Response Code
GET	/api/employees	List all employees	200 OK
GET	/api/employees/{id}	Get employee by ID	200 / 404
POST	/api/employees	Create new employee	201 Created
PUT	/api/employees/{id}	Update employee	200 / 404
DELETE	/api/employees/{id}	Delete employee	200 / 404
GET	/api/employees/search	Search by keyword	200 OK
GET	/api/employees/departments	List all departments	200 OK
GET	/api/health	Health check	200 OK

2.5 Application Profiles — H2 vs Azure SQL

Spring Boot profiles allow different configurations for different environments. We use 'local' (H2) and 'azure' (Azure SQL) profiles:

```
■ application-local.yml

# application-local.yml (H2 - for local dev)

spring:
  datasource:
```



```

url: jdbc:h2:mem:employeeedb;DB_CLOSE_DELAY=-1
driver-class-name: org.h2.Driver
h2:
console:
enabled: true # Access at http://localhost:8080/h2-console
jpa:
hibernate:
ddl-auto: create-drop # Tables auto-created, dropped on exit
show-sql: true # See SQL in console
sql:
init:
data-locations: classpath:data-local.sql # Seed 8 employees

```

■ application-azure.yml

```

# application-azure.yml (Azure SQL – for production)
spring:
datasource:
url: ${AZURE_SQL_URL} # From Key Vault via App Service config
username: ${AZURE_SQL_USERNAME}
password: ${AZURE_SQL_PASSWORD}
driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver
jpa:
database-platform: org.hibernate.dialect.SQLServerDialect
hibernate:
ddl-auto: update # Tables auto-updated (preserves existing data)
show-sql: false

```

2.6 Running Locally

Running Spring Boot Locally

```

# Clone the repo
git clone https://dev.azure.com/yourorg/YourProject/_git/YourRepo
cd employee-management-system
# Run with H2 local profile (NO Azure account needed!)
cd backend
./mvnw spring-boot:run -Dspring-boot.run.profiles=local
# In a new terminal – verify it works
curl http://localhost:8080/api/health
curl http://localhost:8080/api/employees
curl http://localhost:8080/api/employees/1
# Open H2 Console in browser
# URL: http://localhost:8080/h2-console
# JDBC URL: jdbc:h2:mem:employeeedb
# Username: sa | Password: (empty)

```

CHAPTER 3

CRUD Application — Frontend (ReactJS)

React 18 with MUI, React Router, React Hook Form, and Axios for API calls.

3.1 Frontend Project Structure

```
■ Frontend Structure
frontend/
  ■ ■ ■ public/
  ■ ■ ■ ■ index.html
  ■ ■ ■ src/
  ■ ■ ■ ■ App.js # Router + MUI Theme + NavBar
  ■ ■ ■ ■ index.js # ReactDOM entry point
  ■ ■ ■ ■ components/
  ■ ■ ■ ■ ■ EmployeeList.js # Table with search, edit, delete
  ■ ■ ■ ■ ■ EmployeeForm.js # Create & Edit form (React Hook Form)
  ■ ■ ■ ■ ■ EmployeeDetail.js # View single employee details
  ■ ■ ■ ■ services/
  ■ ■ ■ ■ ■ api.js # Axios client + all API calls
  ■ ■ ■ .env.development # REACT_APP_API_URL=/api (proxied)
  ■ ■ ■ .env.production # REACT_APP_API_URL=https://api.azurewebsites.net/api
  ■ ■ ■ package.json
```

3.2 Key Libraries

Library	Version	Purpose
React	18.2	UI framework — components, state, hooks
React Router	6.x	Client-side routing (/ → list, /employees/new → form)
MUI (Material UI)	5.x	Ready-made UI components (Table, Button, TextField, etc.)
Axios	1.x	HTTP client for API calls with interceptors
React Hook Form	7.x	Performant form state + validation (no re-renders on each keystroke)
React Toastify	10.x	Toast notifications (success/error popups)

3.3 Axios API Client

The api.js service file centralises all HTTP calls. It uses Axios interceptors to add logging and standardise error messages:

```
■ api.js – Axios client
// src/services/api.js
const API_BASE_URL = process.env.REACT_APP_API_URL || '/api';
const api = axios.create({
  baseURL: API_BASE_URL,
```

```
headers: { 'Content-Type': 'application/json' },
timeout: 15000,
});
// Response interceptor – standardise errors
api.interceptors.response.use(
(response) => response,
(error) => {
const message = error.response?.data?.message || error.message;
return Promise.reject(new Error(message));
}
);
export const employeeApi = {
getAll: () => api.get('/employees'),
getById: (id)=> api.get(`/employees/${id}`),
create: (d) => api.post('/employees', d),
update:(id,d)=> api.put(`/employees/${id}`, d),
delete: (id)=> api.delete(`/employees/${id}`),
search:(kw) => api.get(`/employees/search?keyword=${encodeURIComponent(kw)}`),
};
```

3.4 Local Development Proxy

In development, React runs on port 3000 and the backend on 8080. The package.json 'proxy' setting automatically forwards API calls from React to Spring Boot, avoiding CORS issues locally:

```
■ Local Proxy Configuration
// package.json
{
"proxy": "http://localhost:8080"
}
// .env.development
REACT_APP_API_URL=/api
// Result: React calls /api/employees
// Proxy forwards to http://localhost:8080/api/employees
// No CORS errors in local dev!
```

3.5 Running Frontend Locally

```
Running React Locally
# Start React (backend must already be running on :8080)
cd frontend
npm install
npm start
# Browser opens: http://localhost:3000
# You'll see the employee list with 8 seed employees!
# Test CRUD operations:
# 1. Click '+ Add Employee' → fill form → Save
# 2. Click the eye icon → view employee details
# 3. Click pencil icon → edit → Save
# 4. Click trash icon → confirm delete
# 5. Type in search box → live search results
```


PART 2

Azure Infrastructure (Portal)

Deploy every resource through the Azure Portal with step-by-step screenshots guidance

Ch 4: Resource Group + SQL

Ch 5: Key Vault

Ch 6: App Services

Ch 7: App Insights

CHAPTER 4

Azure Infrastructure via Portal — Part 1

Resource Group, Networking, Azure SQL Server & Database — step by step with screenshots guidance.

4.1 Create the Resource Group

Everything lives inside a Resource Group. We use a naming convention: rg-{app}-{env}.

1	Search Resource groups In the Azure Portal (portal.azure.com), click the search bar at the top (Ctrl+/) and type 'Resource groups'. Click the service.
2	Click + Create Click the '+ Create' button at the top left of the Resource groups blade.
3	Fill in Basics tab • Subscription: select your subscription • Resource group name: rg-emp-dev • Region: (US) East US
4	Add Tags Click 'Next: Tags'. Add these tags: • Environment = Dev • Project = EmployeeManagement • ManagedBy = Portal
5	Review + Create Click 'Review + create'. Verify all values. Click 'Create'. Wait ~5 seconds for the confirmation notification.
6	Verify Click 'Go to resource group'. You'll see an empty resource group. This is where ALL resources will live.

4.2 Create Azure SQL Server

1	Search SQL servers In the portal search bar, type 'SQL servers' and click the service (not 'SQL databases').
2	Click + Create Click '+ Create'.
3	Basics — Project Details • Subscription: yours • Resource group: rg-emp-dev
4	Basics — Server Details • Server name: emp-sql-dev-001 (must be globally unique!) • Location: East US • Authentication: SQL authentication

5	Set Admin Credentials • Server admin login: sqladmin • Password: EmpM@nag3r2024! (must have upper, lower, digit, symbol) • Confirm password
6	Networking tab • Connectivity method: Public endpoint • Allow Azure services: YES ■ • Add current client IP: YES ■ (for admin access from your PC)
7	Review + Create Click 'Review + create' → 'Create'. Takes 2-3 minutes. Wait for 'Deployment complete' notification.
8	Note the FQDN Go to the SQL server resource. Copy the 'Server name' (e.g., emp-sql-dev-001.database.windows.net). You'll need this later.

4.3 Create Azure SQL Database

1	Open SQL Server Navigate to your emp-sql-dev-001 SQL server resource.
2	Create database Click '+ Create database' in the top toolbar (or go to Databases in the left menu → + Add).
3	Basics • Database name: emp-db-dev • Server: emp-sql-dev-001 (pre-selected) • Want to use SQL elastic pool: No • Workload environment: Development
4	Compute + Storage Click 'Configure database'. • Service tier: Basic • DTUs: 5 • Data max size: 2 GB Click 'Apply'. (This is ~\$5/month, suitable for dev.)
5	Backup Backup storage redundancy: Locally Redundant (cheapest for dev).
6	Review + Create Click 'Review + create' → 'Create'. Takes 1-2 minutes.
7	Test connection Go to the database → Query editor (preview). Login with sqladmin / your password. Run: SELECT @@VERSION — if successful, the database is ready!

■ **NOTE** The SQL Server is the host. The SQL Database is the actual database inside it. One server can host many databases.

CHAPTER 5

Key Vault — Secrets Management

Store ALL secrets in Key Vault. Your app never has a password in its config files.

Azure Key Vault is the security backbone. The Spring Boot backend will READ database credentials directly from Key Vault at startup — no passwords stored in app settings as plaintext!

5.1 Create Azure Key Vault

1	Search Key vaults In the portal search bar, type 'Key vaults' and click the service.
2	Click + Create Click '+ Create'.
3	Basics • Resource group: rg-emp-dev • Key vault name: kv-emp-dev-001 (globally unique, 3-24 chars, alphanumeric + hyphens) • Region: East US • Pricing tier: Standard
4	Access configuration tab • Permission model: Azure role-based access control (RBAC) ← IMPORTANT! Use RBAC, not legacy vault policies. • Enable soft-delete: ON (7-day retention for dev) • Enable purge protection: OFF (for dev — makes it easier to clean up)
5	Networking tab • Allow public access from: All networks (for simplicity in dev; use Private Endpoint in prod)
6	Review + Create Click 'Review + create' → 'Create'. Takes ~20 seconds.

5.2 Assign Yourself Key Vault Administrator

1	Go to Key Vault Navigate to kv-emp-dev-001.
2	Access control (IAM) Click 'Access control (IAM)' in the left menu.
3	Add role assignment Click '+ Add' → 'Add role assignment'.
4	Select role In the Role tab, search for 'Key Vault Administrator'. Select it. Click 'Next'.
5	Select member Click '+ Select members'. Search for your own email/name. Click 'Select'. Click 'Review + assign' twice.

6	Verify Click 'Role assignments' tab. You should see yourself with 'Key Vault Administrator' role.
---	---

5.3 Add Secrets to Key Vault

We'll store 4 secrets: the SQL URL, SQL username, SQL password, and App Insights connection string.

1	Go to Secrets In the Key Vault left menu, click 'Secrets' under Objects.
2	Add SQL URL secret Click '+ Generate/Import'. • Upload options: Manual • Name: AzureSqlUrl • Value: jdbc:sqlserver://emp-sql-dev-001.database.windows.net:1433;database=emp-db-dev;encrypt=true;trustServerCertificate=false;loginTimeout=30 Click 'Create'.
3	Add SQL Username Click '+ Generate/Import'. • Name: AzureSqlUsername • Value: sqladmin Click 'Create'.
4	Add SQL Password Click '+ Generate/Import'. • Name: AzureSqlPassword • Value: EmpM@nag3r2024! Click 'Create'.
5	Add App Insights (after creating it) Name: AppInsightsConnectionString • Value: (copy from Application Insights → Overview → Connection String) Click 'Create'.

5.4 How Key Vault References Work in App Service

Azure App Service has a built-in feature called Key Vault References. You set an App Setting value to a special syntax and Azure automatically fetches the secret from Key Vault:

```
■ Key Vault Reference Syntax
# App Service App Setting value (NOT the actual password!)
AZURE_SQL_PASSWORD = @Microsoft.KeyVault(VaultName=kv-emp-dev-001;SecretName=AzureSqlPassword)
# Azure resolves this at runtime:
# 1. App Service uses its Managed Identity to call Key Vault
# 2. Key Vault returns the secret value
# 3. The env variable AZURE_SQL_PASSWORD = 'EmpM@nag3r2024!'
# 4. Spring Boot reads ${AZURE_SQL_PASSWORD} → gets the real password
# The password NEVER appears in App Service logs or config!
```

■ ■ **WARN** For Key Vault references to work, the App Service MUST have a System-Assigned Managed Identity AND be granted the 'Key Vault Secrets User' RBAC role on the Key Vault. We configure this in Chapter 6.

CHAPTER 6

App Service — Backend & Frontend Deployment

Create App Services for Spring Boot (Java) and React (Node), configure Managed Identity and Key Vault access.

6.1 Create App Service Plan for Backend

1	Search App Service plans In the portal, search 'App Service plans' → '+ Create'.
2	Basics • Resource group: rg-emp-dev • Name: asp-api-dev • Operating System: Linux • Region: East US • Pricing plan: Basic B1 (1 vCPU, 1.75 GB RAM, ~\$13/month)
3	Review + Create Click 'Review + create' → 'Create'.

6.2 Create Backend App Service (Spring Boot)

1	Search App Services Search 'App Services' → '+ Create' → 'Web App'.
2	Basics • Resource group: rg-emp-dev • Name: emp-api-dev (globally unique!) • Publish: Code • Runtime stack: Java 17 • Java web server stack: Java SE (Embedded Web Server) • OS: Linux • Region: East US • App Service Plan: asp-api-dev
3	Review + Create Click 'Review + create' → 'Create'. Takes 1-2 minutes.

6.3 Enable Managed Identity on Backend

1	Go to emp-api-dev App Service Navigate to the App Service resource.
2	Identity In the left menu under Settings, click 'Identity'.
3	System assigned On the 'System assigned' tab, set Status to 'On'. Click 'Save'. Click 'Yes' to confirm.
4	Note the Object ID After enabling, you'll see an 'Object (principal) ID'. Copy it — you need it for the Key Vault role assignment.

6.4 Grant Managed Identity Access to Key Vault

1	Go to Key Vault Navigate to kv-emp-dev-001.
2	Access control (IAM) Click 'Access control (IAM)' → '+ Add' → 'Add role assignment'.
3	Select role Search 'Key Vault Secrets User'. Select it. Click 'Next'.
4	Select managed identity Click '+ Select members'. In the 'Managed identity' dropdown, select 'App Service'. Find emp-api-dev. Click Select. Click 'Review + assign'.
5	Verify The backend App Service can now read all secrets from Key Vault using its own identity — NO password needed!

6.5 Configure Backend App Settings (Key Vault References)

1	Go to emp-api-dev Navigate to the backend App Service.
2	Configuration Click 'Configuration' in the left menu under Settings.
3	Add settings Click '+ New application setting' for each of the following:
4	SPRING_PROFILES_ACTIVE Value: azure (tells Spring Boot to use application-azure.yml)
5	AZURE_SQL_URL Value: @Microsoft.KeyVault(VaultName=kv-emp-dev-001;SecretName=AzureSqlUrl)
6	AZURE_SQL_USERNAME Value: @Microsoft.KeyVault(VaultName=kv-emp-dev-001;SecretName=AzureSqlUsername)
7	AZURE_SQL_PASSWORD Value: @Microsoft.KeyVault(VaultName=kv-emp-dev-001;SecretName=AzureSqlPassword)
8	APPLICATIONINSIGHTS_CONNECTION_STRING Value: @Microsoft.KeyVault(VaultName=kv-emp-dev-001;SecretName=AppInsightsConnectionString)
9	CORS_ALLOWED_ORIGINS Value: https://emp-frontend-dev.azurewebsites.net
10	WEBSITES_PORT / PORT Value: 8080 (Spring Boot default port)

1
1**Save**

Click 'Save' at the top. The app restarts and loads all secrets from Key Vault!

6.6 Verify Key Vault References are Working

1**Check secret status**

In the App Service → Configuration, each KV-reference setting shows a green checkmark ■ if the secret was fetched successfully, or a red ■ if access was denied.

2**Check Managed Identity**

If you see ■, verify: (1) Managed Identity is enabled, (2) RBAC role 'Key Vault Secrets User' is assigned, (3) Key Vault RBAC model is enabled.

6.7 Create Frontend App Service (React)

1**Create App Service Plan**

Create asp-frontend-dev (Linux, Basic B1 or Free F1 for testing).

2**Create App Service**

Name: emp-frontend-dev. Runtime: Node 18 LTS. OS: Linux. Plan: asp-frontend-dev.

3**App Settings for Frontend**

Add setting: REACT_APP_API_URL = https://emp-api-dev.azurewebsites.net/api

4**Note**

Frontend App Service does NOT need a Managed Identity (it calls the backend's public HTTPS URL, not Key Vault directly).

CHAPTER 7

Application Insights & Monitoring

Full observability: request tracing, exceptions, performance, live metrics, and custom KQL queries.

7.1 Create Log Analytics Workspace

1	Search Log Analytics workspaces Search 'Log Analytics workspaces' → '+ Create'.
2	Basics • Resource group: rg-emp-dev • Name: law-emp-dev • Region: East US • Pricing: Pay-as-you-go (first 5GB/month FREE)
3	Review + Create Create. Takes ~30 seconds.

7.2 Create Application Insights

1	Search Application Insights Search 'Application Insights' → '+ Create'.
2	Basics • Resource group: rg-emp-dev • Name: appi-emp-dev • Region: East US • Resource Mode: Workspace-based ■ (newer, recommended) • Log Analytics Workspace: law-emp-dev
3	Review + Create Create. Takes ~20 seconds.
4	Copy Connection String Go to appi-emp-dev → Overview page. Copy the 'Connection String'. Add it to Key Vault as secret 'AppInsightsConnectionString'.

7.3 Useful App Insights Features

Feature	Location in Portal	What it Shows
Live Metrics	appi → Live Metrics	Real-time requests/sec, failures, server health
Failures	appi → Failures	All exceptions and failed requests with stack traces
Performance	appi → Performance	Response time percentiles (P50, P95, P99)
Application Map	appi → Application Map	Visual graph of service dependencies
Availability	appi → Availability	Uptime tests from multiple global regions

Logs (KQL)	appi → Logs	Custom queries on all telemetry data
------------	-------------	--------------------------------------

7.4 Sample KQL Queries

■ App Insights KQL Queries

```
// 1. All failed HTTP requests in the last 1 hour
requests
| where timestamp > ago(1h) and success == false
| project timestamp, name, resultCode, duration
| order by timestamp desc

// 2. Average API response time by endpoint
requests
| where timestamp > ago(24h)
| summarize avg(duration), percentile(duration, 95) by name
| order by avg_duration desc

// 3. All exceptions with stack trace
exceptions
| where timestamp > ago(24h)
| project timestamp, type, outerMessage, operation_Name
| take 50

// 4. Request volume by hour (trend)
requests
| where timestamp > ago(7d)
| summarize count() by bin(timestamp, 1h)
| render timechart
```


PART 3

Azure Infrastructure (CLI)

The same infrastructure deployed entirely with Azure CLI commands

Ch 8: Full CLI Deployment Script

CHAPTER 8

Full CLI Deployment — Step by Step

Deploy the entire infrastructure using Azure CLI commands. Equivalent to Chapters 4-7 but automated.

8.1 Prerequisites

Setup Variables

```
# Install Azure CLI (if not already installed)
# macOS: brew install azure-cli
# Windows: winget install Microsoft.AzureCLI
# Linux: curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

# Login and verify
az login
az account show

# Set variables (change these to match your setup)
export ENV=dev
export LOCATION=eastus
export PREFIX=emp
export RG="rg-${PREFIX}-${ENV}"
export SUFFIX=$(echo $RANDOM | md5sum | head -c 6)
export SQL_SERVER="${PREFIX}-sql-${ENV}-${SUFFIX}"
export SQL_DB="${PREFIX}-db-${ENV}"
export SQL_ADMIN="sqladmin"
export SQL_PASS="EmpM@nag3r2024!" # Use a strong password!
export KV_NAME="kv-${PREFIX}-${ENV}-${SUFFIX}"
export BACKEND_APP="${PREFIX}-api-${ENV}"
export FRONTEND_APP="${PREFIX}-frontend-${ENV}"
export APP_INSIGHTS="appi-${PREFIX}-${ENV}"
export LOG_WS="law-${PREFIX}-${ENV}"
```

8.2 Create Resource Group

Step 1: Resource Group

```
az group create \
--name $RG \
--location $LOCATION \
--tags Environment=$ENV Project=EmployeeManagement
# Verify
az group show --name $RG --output table
```

8.3 Log Analytics + Application Insights

Step 2: Monitoring

```
# Log Analytics Workspace
az monitor log-analytics workspace create \
--resource-group $RG \
--workspace-name $LOG_WS \
--location $LOCATION
LOG_WS_ID=$(az monitor log-analytics workspace show \
```

```
--resource-group $RG --workspace-name $LOG_WS \
--query id -o tsv)
# Application Insights
az monitor app-insights component create \
--app $APP_INSIGHTS \
--resource-group $RG \
--location $LOCATION \
--workspace $LOG_WS_ID
APPI_CONN=$(az monitor app-insights component show \
--app $APP_INSIGHTS --resource-group $RG \
--query connectionString -o tsv)
echo "App Insights Connection String: $APPI_CONN"
```

8.4 Azure SQL Server & Database

```
Step 3: Azure SQL
# Create SQL Server
az sql server create \
--name $SQL_SERVER \
--resource-group $RG \
--location $LOCATION \
--admin-user $SQL_ADMIN \
--admin-password "$SQL_PASS"
# Firewall: allow Azure services
az sql server firewall-rule create \
--resource-group $RG \
--server $SQL_SERVER \
--name AllowAzureServices \
--start-ip-address 0.0.0.0 \
--end-ip-address 0.0.0.0
# Firewall: allow your machine IP for admin
MY_IP=$(curl -s https://api.ipify.org)
az sql server firewall-rule create \
--resource-group $RG --server $SQL_SERVER \
--name AllowMyIP \
--start-ip-address $MY_IP --end-ip-address $MY_IP
# Create database (Basic tier = 5 DTU, 2GB)
az sql db create \
--resource-group $RG \
--server $SQL_SERVER \
--name $SQL_DB \
--edition Basic \
--capacity 5
# Get the SQL Server FQDN
SQL_FQDN=$(az sql server show \
--name $SQL_SERVER --resource-group $RG \
--query fullyQualifiedDomainName -o tsv)
SQL_URL="jdbc:sqlserver://${SQL_FQDN}:1433;database=${SQL_DB};encrypt=true;trustServerCertificate=false;
loginTimeout=30"
echo "SQL URL: $SQL_URL"
```

8.5 Key Vault + Secrets

```
Step 4: Key Vault
```

```
# Create Key Vault with RBAC authorization
az keyvault create \
--name $KV_NAME \
--resource-group $RG \
--location $LOCATION \
--enable-rbac-authorization true \
--soft-delete-retention-days 7
KV_ID=$(az keyvault show --name $KV_NAME --resource-group $RG --query id -o tsv)
# Grant YOURSELF Key Vault Administrator so you can add secrets
CURRENT_USER=$(az account show --query user.name -o tsv)
az role assignment create \
--assignee $CURRENT_USER \
--role 'Key Vault Administrator' \
--scope $KV_ID
# Store all 4 secrets
az keyvault secret set --vault-name $KV_NAME \
--name AzureSqlUrl --value "$SQL_URL"
az keyvault secret set --vault-name $KV_NAME \
--name AzureSqlUsername --value "$SQL_ADMIN"
az keyvault secret set --vault-name $KV_NAME \
--name AzureSqlPassword --value "$SQL_PASS"
az keyvault secret set --vault-name $KV_NAME \
--name AppInsightsConnectionString --value "$APPI_CONN"
echo '■ All secrets stored in Key Vault'
# Verify
az keyvault secret list --vault-name $KV_NAME --output table
```

8.6 App Service Plans & Web Apps

```
Step 5: App Services
# Backend App Service Plan (Java)
az appservice plan create \
--name asp-api-dev \
--resource-group $RG \
--location $LOCATION \
--sku B1 --is-linux
# Backend Web App (Java 17)
az webapp create \
--resource-group $RG \
--plan asp-api-dev \
--name $BACKEND_APP \
--runtime 'JAVA:17-java17'
# Enable Managed Identity on backend
az webapp identity assign \
--resource-group $RG --name $BACKEND_APP
BACKEND_MI=$(az webapp identity show \
--resource-group $RG --name $BACKEND_APP \
--query principalId -o tsv)
echo "Backend Managed Identity: $BACKEND_MI"
# Frontend App Service Plan (Node)
az appservice plan create \
--name asp-frontend-dev \
--resource-group $RG \
--location $LOCATION \
```

```
--sku B1 --is-linux
# Frontend Web App (Node 18)
az webapp create \
--resource-group $RG \
--plan asp-frontend-dev \
--name $FRONTEND_APP \
--runtime 'NODE:18-lts'
```

8.7 Configure Backend App Settings (Key Vault References)

Step 5: App Settings

```
FRONTEND_URL="https://${FRONTEND_APP}.azurewebsites.net"
az webapp config appsettings set \
--resource-group $RG --name $BACKEND_APP \
--settings \
SPRING_PROFILES_ACTIVE=azure \
"AZURE_SQL_URL=@Microsoft.KeyVault(VaultName=${KV_NAME};SecretName=AzureSqlUrl)" \
"AZURE_SQL_USERNAME=@Microsoft.KeyVault(VaultName=${KV_NAME};SecretName=AzureSqlUsername)" \
"AZURE_SQL_PASSWORD=@Microsoft.KeyVault(VaultName=${KV_NAME};SecretName=AzureSqlPassword)" \
"APPLICATIONINSIGHTS_CONNECTION_STRING=@Microsoft.KeyVault(VaultName=${KV_NAME};SecretName=AppInsightsCo
nnectionString)" \
CORS_ALLOWED_ORIGINS=$FRONTEND_URL \
WEBSITES_PORT=8080 \
PORT=8080
# Configure frontend
BACKEND_URL="https://${BACKEND_APP}.azurewebsites.net"
az webapp config appsettings set \
--resource-group $RG --name $FRONTEND_APP \
--settings \
REACT_APP_API_URL="${BACKEND_URL}/api" \
WEBSITE_NODE_DEFAULT_VERSION=18
```

8.8 Grant Managed Identity Key Vault Access (RBAC)

Step 7: RBAC & HTTPS

```
# Grant backend Managed Identity the 'Key Vault Secrets User' role
az role assignment create \
--assignee-object-id $BACKEND_MI \
--assignee-principal-type ServicePrincipal \
--role 'Key Vault Secrets User' \
--scope $KV_ID
echo '■ Backend can now read all Key Vault secrets!'
# Enable HTTPS only on both apps
az webapp update --resource-group $RG --name $BACKEND_APP --https-only true
az webapp update --resource-group $RG --name $FRONTEND_APP --https-only true
# Print summary
echo "Frontend: https://${FRONTEND_APP}.azurewebsites.net"
echo "Backend: https://${BACKEND_APP}.azurewebsites.net/api/health"
echo "Key Vault: $KV_NAME"
```


PART 4

Bicep Infrastructure as Code

One declarative template to deploy everything consistently

Ch 9: Bicep Template Explained

Ch 10: Deploy with Bicep

Bicep Template — Full Explanation

9.1 What is Bicep?

Bicep vs CLI	Bicep (IaC)	Azure CLI (Imperative)
Style	Declarative (describe end state)	Imperative (step by step)
Idempotent	■ Run multiple times safely	■■ Must handle 'already exists'
Source control	■ Version control infrastructure	Less structured
Rollback	Via ARM complete mode	Manual
Best for	Repeatable environments	Quick ad-hoc tasks

9.2 Bicep File Structure

```
# main.bicep - Structure
// main.bicep - annotated structure
targetScope = 'resourceGroup' // Deploy to a resource group

// Parameters (inputs)
@description('Environment: dev, staging, prod')
@allowed(['dev', 'staging', 'prod'])
param environment string = 'dev'

@secure() // Marked secure – not logged, not shown in portal
param sqlAdminPassword string

// Variables (computed values)
var prefix = 'emp'

var suffix = uniqueString(resourceGroup().id) // Auto-unique suffix
var names = {
sqlServer: '${prefix}-sql-${environment}-${take(suffix, 6)}'
keyVault: 'kv-${prefix}-${environment}-${take(suffix, 6)}'
// ... more names
}

// Resources
resource sqlServer 'Microsoft.Sql/servers@2023-05-01-preview' = {
name: names.sqlServer
location: location
properties: { ... }
}

// Child resources
resource sqlDatabase 'Microsoft.Sql/servers/databases@2023-05-01-preview' = {
parent: sqlServer // Child of sqlServer
```

```
name: names.sqlDatabase
// ...
}

// █ Outputs ████████████████████████████████████████████████████
output frontendUrl string = 'https://${frontendApp.properties.defaultHostName}'
```

9.3 Key Bicep Concepts Explained

- **@secure() parameter** — Tells Bicep the value is sensitive. Not logged or shown in deployment history.
- **uniqueString()** — Generates a deterministic unique string from the resource group ID. Same RG always produces the same string, so re-runs don't create duplicates.
- **parent: resource** — Defines resource hierarchy (SQL database is a child of SQL server).
- **Managed Identity in Bicep** — Setting identity: { type: 'SystemAssigned' } enables the identity. Then use the .identity.principalId property for RBAC.
- **RBAC role assignment** — Uses subscriptionResourceId() to reference the built-in role definition.
- **Key Vault references in Bicep** — The App Setting values are set to the @Microsoft.KeyVault() reference syntax directly in the template.

9.4 Managed Identity + RBAC in Bicep

```

■ Bicep – Managed Identity + RBAC
// Enable Managed Identity on App Service
resource backendApp 'Microsoft.Web/sites@2023-01-01' = {
  name: names.backendApp
  identity: { type: 'SystemAssigned' } // ← This enables it!
  properties: { ... }
}

// Grant it 'Key Vault Secrets User' role
var kvSecretsUserRoleId = '4633458b-17de-408a-b874-0445c86b69e6'
resource kvRoleAssignment 'Microsoft.Authorization/roleAssignments@2022-04-01' = {
  name: guid(keyVault.id, backendApp.id, kvSecretsUserRoleId)
  scope: keyVault // ← Scoped to Key Vault only
  properties: {
    roleDefinitionId: subscriptionResourceId(
      'Microsoft.Authorization/roleDefinitions', kvSecretsUserRoleId)
    principalId: backendApp.identity.principalId // ← The MI's ID
    principalType: 'ServicePrincipal'
  }
}

```


CHAPTER 10

Deploy & Manage with Bicep

Run the Bicep template, verify outputs, and update infrastructure.

10.1 Deploy the Bicep Template

```
Deploy Bicep Template
# Step 1: Create the resource group first (Bicep deploys INTO a RG)
az group create --name rg-emp-dev --location eastus

# Step 2: Validate the template (dry-run, no resources created)
az deployment group validate \
--resource-group rg-emp-dev \
--template-file infrastructure/bicep/main.bicep \
--parameters environment=dev sqlAdminPassword='EmpM@nag3r2024!'

# Step 3: What-if (shows exactly what will be created/changed)
az deployment group what-if \
--resource-group rg-emp-dev \
--template-file infrastructure/bicep/main.bicep \
--parameters environment=dev sqlAdminPassword='EmpM@nag3r2024!'

# Step 4: Actually deploy!
az deployment group create \
--resource-group rg-emp-dev \
--template-file infrastructure/bicep/main.bicep \
--parameters environment=dev \
sqlAdminPassword='EmpM@nag3r2024!' \
--name 'deploy-emp-dev-001'

# Step 5: Check outputs from the deployment
az deployment group show \
--resource-group rg-emp-dev \
--name deploy-emp-dev-001 \
--query properties.outputs
```

10.2 Using the Parameter File

```
Using Parameter File
# Use the .bicepparam file instead of inline params
# Edit infrastructure/bicep/main.bicepparam first,
# then deploy:
az deployment group create \
--resource-group rg-emp-dev \
--template-file infrastructure/bicep/main.bicep \
--parameters infrastructure/bicep/main.bicepparam \
--parameters sqlAdminPassword='EmpM@nag3r2024!'

# Note: sqlAdminPassword overrides the empty value in param file
```

10.3 Deploy to Different Environments

```
Multi-Environment Deployment
# Create staging environment
```

```
az group create --name rg-emp-staging --location eastus
az deployment group create \
--resource-group rg-emp-staging \
--template-file infrastructure/bicep/main.bicep \
--parameters environment=staging sqlAdminPassword='StagP@ss2024!'
# Create production environment (larger SKUs automatically!)
az group create --name rg-emp-prod --location eastus
az deployment group create \
--resource-group rg-emp-prod \
--template-file infrastructure/bicep/main.bicep \
--parameters environment=prod sqlAdminPassword='ProdP@ss2024!'
# Production gets S2 App Service Plan and S1 SQL automatically
# because the Bicep template conditions on the environment param!
```


PART 5

Azure DevOps Pipelines

Automated CI/CD for frontend and backend with Blue-Green deployment

Ch 11: Pipeline Setup

Ch 12: Frontend Pipeline

Ch 13: Backend Pipeline

Ch 14: Blue-Green

CHAPTER 11

Pipeline Setup — Service Connections & Variable Groups

Configure Azure DevOps before running the pipelines.

11.1 Create the Service Connection

A Service Connection lets Azure Pipelines authenticate to your Azure subscription to deploy resources. It uses a Service Principal under the hood.

1	Open Azure DevOps Go to dev.azure.com. Open your project (or create one).
2	Project Settings Click 'Project settings' (gear icon at bottom left).
3	Service connections Under Pipelines section, click 'Service connections'.
4	New service connection Click '+ New service connection' → 'Azure Resource Manager' → 'Next'.
5	Authentication method Select 'Service principal (automatic)'. This creates a Service Principal automatically with Contributor access.
6	Configure • Scope level: Subscription • Subscription: select yours • Resource group: leave blank (subscription-level is fine) • Service connection name: sc-azure-emp • Check 'Grant access permission to all pipelines' Click 'Save'.
7	Verify The service connection appears in the list with a green checkmark. Test it by clicking 'Verify connection'.

11.2 Create Variable Groups

Variable groups store shared variables and secrets used across pipelines.

1	Open Library In Azure DevOps, click 'Pipelines' → 'Library'.
2	Create variable group Click '+ Variable group'. Name: azure-global-vars.

3	Add variables Click '+ Add variable' for each: • RESOURCE_GROUP = rg-emp-dev • RG_PROD = rg-emp-prod • SERVICE_CONNECTION = sc-azure-emp
4	Pipeline permissions Click 'Pipeline permissions' → '+ Add' → select your pipelines. Click Save.

11.3 Create Pipeline Environments with Approval Gates

1	Create environments In Azure DevOps, click 'Pipelines' → 'Environments' → '+ New environment'.
2	frontend-dev Name: frontend-dev. Resource: None. Click Create.
3	frontend-prod Name: frontend-prod. Resource: None. Click Create.
4	Add approval to frontend-prod Click 'frontend-prod' → 'Approvals and checks' tab → '+' → 'Approvals'. • Approvers: add your email • Instructions: 'Verify dev deployment before approving' • Timeout: 24 hours Click Create.
5	Repeat for backend Create: backend-dev, backend-prod. Add approval gate to backend-prod.

11.4 Create the Pipelines

1	New Pipeline Click 'Pipelines' → '+ New pipeline'.
2	Connect code Choose 'Azure Repos Git' (or GitHub). Select your repository.
3	Existing YAML Select 'Existing Azure Pipelines YAML file'. Path: /pipelines/azure-pipelines-frontend.yml.
4	Save and run Click 'Save and run'. The first run builds and deploys to dev!
5	Repeat for backend Create another pipeline using /pipelines/azure-pipelines-backend.yml.

CHAPTER 12

React Frontend CI/CD Pipeline — Deep Dive

Every step of the 3-stage frontend pipeline explained in detail.

12.1 Pipeline Trigger

```

■ Pipeline Trigger
trigger:
branches:
include: [main, develop, release/*]
paths:
include: [frontend/**] # Only runs when frontend/ changes!
pr:
branches:
include: [main, develop]
paths:
include: [frontend/**]

```

The path filter is critical — backend changes don't trigger the frontend pipeline, and vice versa. This saves build minutes and reduces noise.

12.2 Stage 1 — Build & Test

- **NodeTool@0** — Installs the exact Node.js version specified (18.x). Ensures consistency across all agent runs.
- **npm ci** — 'Clean install' (faster than npm install, uses package-lock.json exactly, fails if lock file is out of date). Always use in CI!
- **npm test -- --coverage --watchAll=false** — Runs Jest tests in CI mode. --watchAll=false is required in CI, otherwise it waits forever.
- **PublishTestResults@2** — Uploads test results to Azure DevOps so you see pass/fail counts in the pipeline summary.
- **npm run build** — Creates the optimised production build in the build/ folder (minified JS, hashed filenames).
- **PublishPipelineArtifact@1** — Saves the build/ folder as a pipeline artifact named 'frontend-build'. Later stages download this instead of rebuilding.

12.3 Stage 2 — Deploy to Dev

- **Rebuild with correct API URL** — The Dev stage rebuilds React with REACT_APP_API_URL pointing to the Dev backend. This is necessary because React bakes env vars at build time (not runtime).
- **Zip the build folder** — Azure App Service zip deploy is the fastest deployment method. We exclude .map files to reduce size.
- **az webapp deployment source config-zip** — Uploads and extracts the zip to the App Service's /home/site/wwwroot directory.
- **Configure SPA routing** — Sets the startup command to pm2 serve with --spa flag. This makes the Node server return index.html for all routes, enabling React Router to work correctly.
- **Smoke test** — After deployment, makes an HTTP request and verifies the response code is 200-399. Fails the pipeline if the site is down.

12.4 Stage 3 — Deploy to Production

- **Condition** — Only runs when deploying from the 'main' branch (not feature branches or develop).
- **Environment: frontend-prod** — This triggers the manual approval gate. The pipeline PAUSES here until an approver clicks 'Approve' in Azure DevOps.
- **GENERATE_SOURCEMAP=false** — Disables source maps in production. Source maps expose your source code to end users (visible in browser DevTools).
- **Separate resource group** — The production deployment targets RG_PROD (rg-emp-prod), a completely separate environment from dev.

CHAPTER 13

Spring Boot Backend CI/CD Pipeline — Deep Dive

Maven build, H2 tests, JAR packaging, and Blue-Green production deployment.

13.1 Stage 1 — Maven Build, Test & Package

- **JavaToolInstaller@0** — Ensures Java 17 is available on the agent. Azure agents come with Java pre-installed; this task selects the correct version.
- **Maven test with local profile** — Runs `mvn test -Dspring.profiles.active=local`. This activates H2 profile — the tests run against the in-memory database, no Azure SQL needed!
- **PublishTestResults@2** — Uploads Surefire XML test reports to the pipeline UI.
- **mvn package -DskipTests** — Creates the executable JAR. We skip tests because they already ran in the previous step.
- **CopyFiles + PublishArtifact** — The JAR is ~50MB. We publish it once and all deploy stages download the same artifact — no rebuilding.

13.2 Stage 2 — Deploy to Dev

- **AzureWebApp@1 task** — This is the simplest way to deploy a JAR to App Service. It handles authentication via the service connection and performs a zip deploy of the JAR.
- **startUpCommand** — Sets the Java command used to start the app: `java -jar app.jar --spring.profiles.active=azure`. This activates the Azure SQL profile.
- **Health check with retry** — After deployment, the app needs ~60 seconds to start (Spring Boot startup + database migrations). The script retries 5 times with 15-second intervals.
- **Integration tests** — Makes real HTTP calls to the deployed API: `GET /employees (200)`, `GET /employees/999 (404)`. These verify the app is truly working, not just responding with 200.

13.3 Stage 3 — Blue-Green Production Deployment

Blue-Green deployment eliminates production downtime. Here's how it works with App Service Deployment Slots:

- **Step 1: Deploy to staging slot** — The new version is deployed to `emp-api-prod-staging.azurewebsites.net`. This is a separate instance running the new code while production still runs the old code.
- **Step 2: Test staging slot** — Run the health check against the staging URL. If it fails, we abort — production is untouched!
- **Step 3: Swap slots** — Azure instantly swaps the staging and production slots. Users are now hitting the new code. The old code moves to the staging slot.
- **Step 4: Post-swap health check** — If production fails after the swap, we swap back (rollback). This is the emergency rollback mechanism.

■ Blue-Green Swap Commands

```
# The swap command (atomic, zero downtime):
az webapp deployment slot swap \
--resource-group $RG \
--name emp-api-prod \
--slot staging \
--target-slot production
```

```
# If production breaks, rollback in seconds:  
az webapp deployment slot swap \  
--resource-group $RG \  
--name emp-api-prod \  
--slot staging \  
--target-slot production # Swaps back to old version
```

CHAPTER 14

Blue-Green Deployment Strategy

Zero-downtime deployments with instant rollback capability.

14.1 Why Blue-Green?

Deployment Type	Downtime	Rollback Time	Risk
In-place (stop, deploy, start)	30-120 seconds	Redeploy (5+ min)	High
Rolling update	Near-zero	Partial rollback only	Medium
Blue-Green (slots swap)	Zero	< 5 seconds	Low

14.2 Slot Warm-Up

Before the swap, App Service 'warms up' the staging slot by sending requests to it. This ensures the new version is fully started before it receives production traffic. Configure this in App Service → Configuration → General settings → Auto Swap or use the pipeline's sleep/health-check approach.

14.3 Setting Up Deployment Slots via CLI

```
Deployment Slots CLI
# Create staging slot for backend (requires Standard+ plan)
az webapp deployment slot create \
--resource-group rg-emp-prod \
--name emp-api-prod \
--slot staging \
--configuration-source emp-api-prod
# The slot gets its own URL:
# https://emp-api-prod-staging.azurewebsites.net
# List all slots
az webapp deployment slot list \
--resource-group rg-emp-prod \
--name emp-api-prod \
--output table
# Perform a manual swap
az webapp deployment slot swap \
--resource-group rg-emp-prod \
--name emp-api-prod \
--slot staging
# Emergency rollback (swap back)
az webapp deployment slot swap \
--resource-group rg-emp-prod \
--name emp-api-prod \
--slot staging
```


PART 6

Configuration & Secrets

Deep dive into Key Vault integration and Spring Boot profiles

Ch 15: Key Vault Integration

Ch 16: Configuration Profiles

CHAPTER 15

Key Vault Integration — Portal, CLI & Code

The complete secret lifecycle: store, read, rotate, revoke.

15.1 The Complete Secret Flow

- 1. **Secret created** in Key Vault (via Portal, CLI, or Bicep).
- 2. **App Service config** has a Key Vault Reference: @Microsoft.KeyVault(VaultName=...)
- 3. **At App Service startup**, Azure reads the reference and fetches the secret using the App Service's Managed Identity.
- 4. **Spring Boot starts** and reads environment variables — \${AZURE_SQL_PASSWORD} = 'actual-password'.
- 5. **Secret is cached** for 24 hours. To force refresh: restart the App Service or update the secret version.

15.2 Managing Secrets via CLI

```
Key Vault Secret Lifecycle
KV='kv-emp-dev-001'
# List all secrets
az keyvault secret list --vault-name $KV --output table
# Get a secret value
az keyvault secret show --vault-name $KV --name AzureSqlPassword \
--query value -o tsv
# Update/rotate a secret (creates a new version)
az keyvault secret set --vault-name $KV \
--name AzureSqlPassword \
--value 'NewPassword2024!'
# After rotating: restart App Service to pick up new value
az webapp restart --resource-group rg-emp-dev --name emp-api-dev
# List secret versions (full history)
az keyvault secret list-versions --vault-name $KV \
--name AzureSqlPassword --output table
# Disable a secret (revoke access without deleting)
az keyvault secret set-attributes --vault-name $KV \
--name AzureSqlPassword --enabled false
# Soft-delete (recoverable for 7 days in dev config)
az keyvault secret delete --vault-name $KV --name AzureSqlPassword
# Restore deleted secret
az keyvault secret recover --vault-name $KV --name AzureSqlPassword
```

15.3 Reading Secrets in Spring Boot Code

Spring Boot reads secrets as regular environment variables. The @Value annotation injects them:

```
■ Reading Secrets in Java
// You DON'T need Azure SDK to read Key Vault secrets!
// Spring Boot reads the env var that App Service injected from KV.
// In application-azure.yml:
spring:
```

```
datasource:
password: ${AZURE_SQL_PASSWORD} // Azure resolved this from KV
// Or inject into a @Component:
@Value("${AZURE_SQL_PASSWORD:defaultValue}")
private String dbPassword;
// Or use @ConfigurationProperties for grouped config
@ConfigurationProperties(prefix = "spring.datasource")
public class DatabaseConfig {
private String url; // from AZURE_SQL_URL
private String username; // from AZURE_SQL_USERNAME
private String password; // from AZURE_SQL_PASSWORD
}
```

CHAPTER 16

Application Configuration — Profiles Deep Dive

How Spring Boot profiles enable H2 locally and Azure SQL in production.

16.1 Profile Activation

Method	Command/Config	Use When
Maven command line	mvnw spring-boot:run -Dspring-boot.run.profiles=local	Local development
JAR command line	java -jar app.jar --spring.profiles.active=azure	Production JAR
Environment variable	SPRING_PROFILES_ACTIVE=azure	Container / App Service
application.yml	spring.profiles.active: \${SPRING_PROFILES_ACTIVE:local}	Default fallback

16.2 Profile Loading Order

Spring Boot merges configuration files in this priority order (higher = wins):

- 1. **application.yml** — Base config, always loaded
- 2. **application-{profile}.yml** — Profile-specific, overrides base
- 3. **Environment variables** — Override everything (SPRING_DATASOURCE_URL etc.)
- 4. **Command-line arguments** — Highest priority

16.3 Complete Configuration Reference

```
■ application.yml
# application.yml (base – always loaded)
spring:
  application:
    name: employee-management-api
    profiles:
      active: ${SPRING_PROFILES_ACTIVE:local} # Default to local
    server:
      port: ${PORT:8080}
    management:
      endpoints:
        web:
          exposure:
            include: health,info,metrics

■ application-local.yml
# application-local.yml (H2 profile)
spring:
  datasource:
    url: jdbc:h2:mem:employeedb;DB_CLOSE_DELAY=-1
```



```
driver-class-name: org.h2.Driver
username: sa
password: ''
h2.console.enabled: true
jpa:
hibernate.ddl-auto: create-drop
show-sql: true
sql.init.data-locations: classpath:data-local.sql
```

■ application-azure.yml

```
# application-azure.yml (Azure SQL profile)
spring:
datasource:
url: ${AZURE_SQL_URL} # From Key Vault via App Setting
username: ${AZURE_SQL_USERNAME}
password: ${AZURE_SQL_PASSWORD}
driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver
hikari:
maximum-pool-size: 10
connection-timeout: 30000
jpa:
database-platform: org.hibernate.dialect.SQLServerDialect
hibernate.ddl-auto: update # Preserves existing data!
show-sql: false
```


PART 7

Verification & Troubleshooting

Test everything works end-to-end and fix common problems

Ch 17: End-to-End Verification

Ch 18: Common Issues & Fixes

CHAPTER 17

End-to-End Testing & Verification

How to verify the entire stack is working correctly after deployment.

17.1 Verify Backend API

Backend API Verification

```
BACKEND='https://emp-api-dev.azurewebsites.net'

# Health check
curl $BACKEND/api/health | python3 -m json.tool
# Expected: {status: UP, profile: azure}

# Get all employees
curl $BACKEND/api/employees | python3 -m json.tool
# Expected: [] or list of employees

# Create an employee
curl -X POST $BACKEND/api/employees \
-H 'Content-Type: application/json' \
-d '{"firstName":"Test","lastName":"User","email":"test@example.com",
"department":"Engineering","position":"Developer","salary":80000}'
# Expected: 201 Created with the new employee object

# Get employee by ID (use ID from previous response)
curl $BACKEND/api/employees/1 | python3 -m json.tool

# Update employee
curl -X PUT $BACKEND/api/employees/1 \
-H 'Content-Type: application/json' \
-d '{"firstName":"Updated","lastName":"User","email":"test@example.com",
"department":"Product","position":"PM","salary":90000}'

# Delete employee
curl -X DELETE $BACKEND/api/employees/1

# Search
curl "$BACKEND/api/employees/search?keyword=Engineering"
```

17.2 Verify Key Vault Integration

Key Vault Verification

```
# Check if App Service is reading KV secrets correctly
az webapp config appsettings list \
--resource-group rg-emp-dev \
--name emp-api-dev \
--query '[_contains(value, `KeyVault`)]' \
--output table
# Should show green (resolved) status for KV references

# Check App Service logs for startup errors
az webapp log tail \
--resource-group rg-emp-dev \
--name emp-api-dev
# Look for: 'Started AzureDemoApplication in X.XXX seconds'
# Errors like 'Login failed for user' mean DB credentials are wrong
```

17.3 Verify Application Insights

1	Open App Insights Go to appi-emp-dev in the portal.
2	Live Metrics Click 'Live Metrics'. Make some API calls. You'll see requests appearing in real time!
3	Check logs Click 'Logs'. Run: requests take 10 — see your test API calls logged.
4	Verify exceptions Run: exceptions take 10 — if empty, great! No errors.

CHAPTER 18

Common Issues & Fixes

The most common deployment problems and how to solve them.

Problem	Symptom	Cause	Fix
KV Reference ■ in App Settings	Red X next to setting	No RBAC role on Key Vault	Grant 'Key Vault Secrets User' to App Service MI
Spring Boot won't start	503 Service Unavailable	Missing env var / wrong profile	Check SPRING_PROFILES_ACTIVE=azure; check App logs
CORS error in browser	Network tab: CORS policy	CORS_ALLOWED_ORIGINS wrong	Set to exact frontend URL (https://...azurewebsites.net)
React shows blank page	Browser console: Cannot GET /employees	SPA routing not configured	Set startup command to pm2 serve --spa
SQL Login failed	500 error on any API call	Wrong password / firewall	Check KV secret value; check SQL firewall rules
Pipeline: AzureWebApp@1 fails	'Service Connection not found'	SC not created or no permission	Create SC in Project Settings; grant pipeline access
H2 console shows empty tables	No data in H2	data-local.sql not loading	Check spring.sql.init.mode=always in local profile

18.1 Check App Service Logs

App Service Logging

```
# Enable logging first
az webapp log config \
--resource-group rg-emp-dev \
--name emp-api-dev \
--application-logging filesystem \
--level information
# Stream live logs
az webapp log tail \
--resource-group rg-emp-dev \
--name emp-api-dev
# Download logs
az webapp log download \
--resource-group rg-emp-dev \
--name emp-api-dev \
--log-file ./applogs.zip
```

18.2 Cleanup All Resources

```
cleanup
# Delete entire dev environment (saves money!)
az group delete --name rg-emp-dev --yes --no-wait
# Delete prod environment
az group delete --name rg-emp-prod --yes --no-wait
# Verify deletion
az group list --query '[?contains(name, `emp`)]' --output table
```